End-to-End Encrypted Messaging Using Signal-Protocol

Dhiraj Shelke

May 5, 2026

Course: CS 588 **Assignment:** End-to-end encrypted messaging (Signal protocol)

1 Introduction

Networked messaging systems increasingly assume an untrusted transport: the service may store and forward data, but must not require message plaintext. Signal's ecosystem popularized asynchronous prekey bundles, X3DH for first contact, and the Double Ratchet for ongoing secrecy and recovery from reordering.

This project presents a complete browser-based messenger with user registration, 1:1 encrypted sessions, bidirectional send/receive, conversation recovery after reload, and partner listing. Plaintext and long-term private key material remain on client devices. The interface also exposes protocol-state diagnostics (handshake and ratchet summaries) so security behavior can be observed directly.

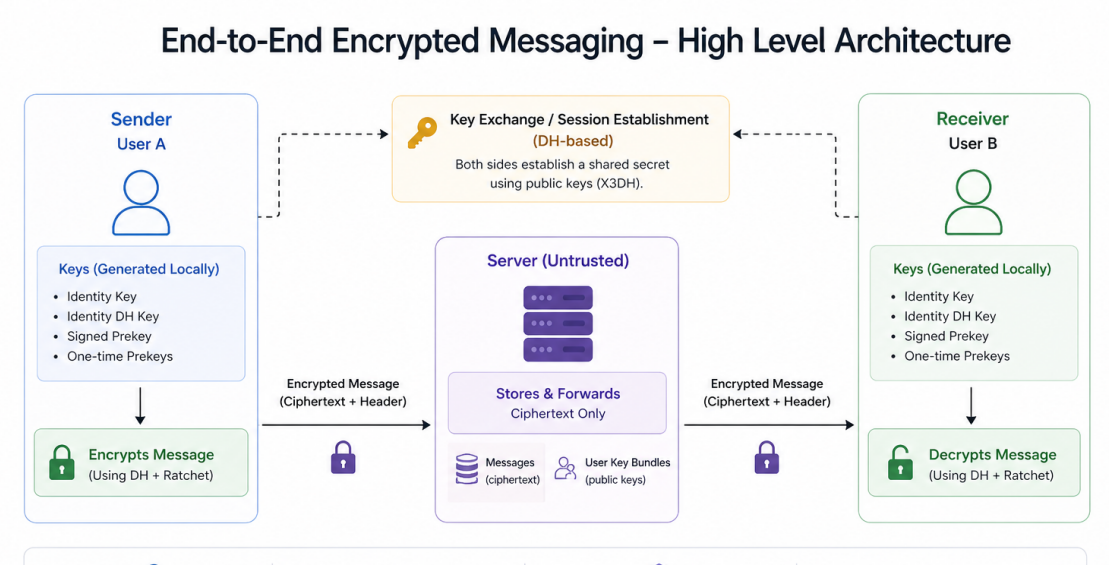


Figure 1: System architecture with two browser clients, an untrusted relay, and ciphertext/public-key persistence.

2 System architecture

2.1 Components and stack

- **Web client:** Next.js with React and TypeScript, registration, chat, and optional diagnostics over session and ratchet state. Private keys and ratchet serialization live in ordinary browser storage for this educational deployment.
- **Relay service:** Python, FastAPI, served with Uvicorn, REST endpoints for registering bundles, fetching or reserving prekeys (consuming optional one-time keys), posting ciphertext, polling undelivered traffic, and reconciling conversations.
- **Persistence:** Lightweight JSON files on the server retain published public material and message records, no relational layer is required for the coursework scope.
- **Integration:** During local development the SPA often talks to the API via same-origin HTTP proxying, production-style separation would rely on explicit cross-origin configuration.

2.2 Logical data flow

```
[Client A]  --encrypt (X3DH + Double Ratchet)-->  ciphertext envelope
      |                                                  |
      v                                                  v
[Relay API]   store / retrieve public keys only   --->  [Client B]
                                                          decrypt locally
```

The relay never performs pairwise decryption, it moves opaque blobs and routing metadata.

2.3 Representative API semantics

The API supports health checks, registration with a public bundle and update credential, bundle retrieval and reserve semantics (which may consume a one-time prekey), ciphertext submission, inbox polling with delivery updates, and conversation/chat-list queries for synchronization. Request and response payloads are validated at the service boundary using structured schemas.

2.4 Trust model

Boundary	Interpretation
Trusted for secrecy of content	Browser runtime executing the cryptography, user device cleanliness is assumed for evaluation.
Untrusted for plaintext	Relay and disk backups of server state, attackers gain metadata, public keys, and ciphertext only under the stated threat model.
Intentionally visible	Participant identifiers, pairwise addressing, timestamps implied by ordering, message sizes, conversation listing.

Table 1: Simplified trust boundaries for the coursework threat model.

Metadata hiding (who contacts whom beyond what the relay must see) is out of scope.

2.5 Design rationale

Client-side cryptography limits exposure if the relay is breached. Hand-implemented X3DH and Double Ratchet logic (on top of vetted primitives) favors auditing over opaque dependencies. JSON persistence keeps the server a thin conduit rather than a second trust domain for message content.

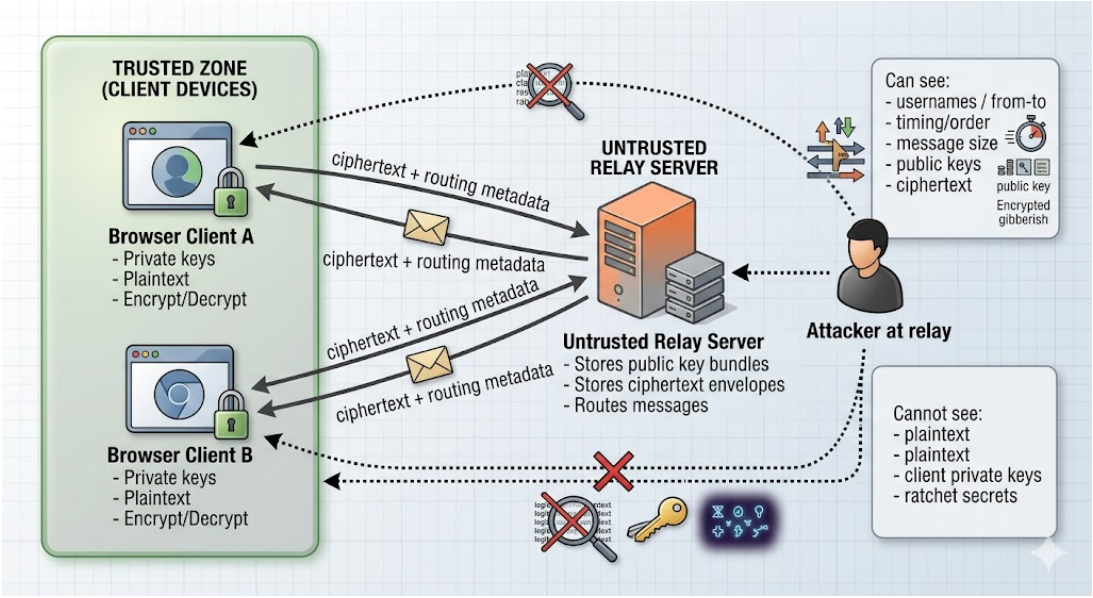


Figure 2: Trust-boundary model: trusted client endpoints versus untrusted relay with metadata and ciphertext visibility.

3 Cryptographic design

3.1 Primitives and libraries

- **Elliptic-curve DH and signatures:** X25519 and Ed25519 via the TweetNaCl family in JavaScript.
- **Key derivation:** HKDF-SHA256 (RFC 5869) using Web Crypto HMAC primitives.
- **Authenticated encryption:** XSalsa20-Poly1305 ("secretbox") for message bodies, nonces and optional associated data binding follow the in-browser implementation.

3.2 X3DH session establishment

The initiator retrieves the responder's identity DH key, signed prekey (with Ed25519 signature), and optional one-time prekey, then performs three or four DH exchanges, concatenates the results, and expands with HKDF. The first protected message carries the initiator's ephemeral DH public key and references to the prekeys used so the responder can recompute the same secret asynchronously. The responder verifies the signed prekey, derives shared material, and bootstraps the Double Ratchet in lockstep.

3.3 Double Ratchet messaging

After bootstrap, root and chain keys derive per-message keys. Each ciphertext includes a compact header: peer ratchet DH public key, send counter n, and prior chain length pn for bookkeeping. Symmetric ratchet steps advance chains per direction, asymmetric DH ratchet steps occur when the peer publishes a fresh ratchet key. Out-of-order delivery is handled with a bounded cache of skipped message keys (constant cap, e.g. two hundred steps in this deployment). Decryption failures indicate tampering or inconsistent state relative to AEAD verification.

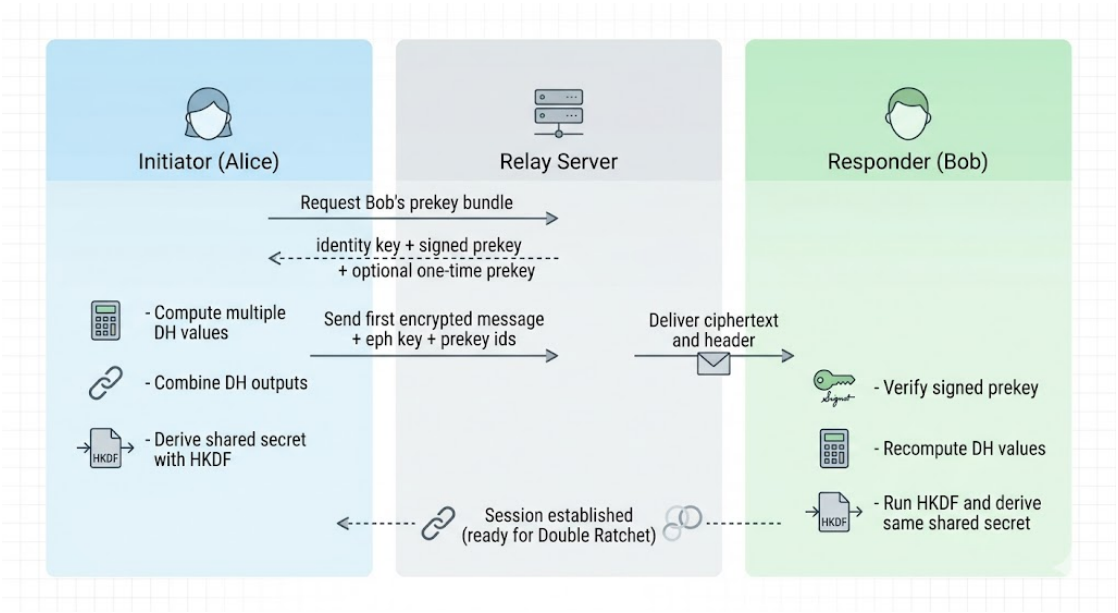


Figure 3: X3DH session setup flow: bundle retrieval, multi-DH, HKDF derivation, and first encrypted message metadata.

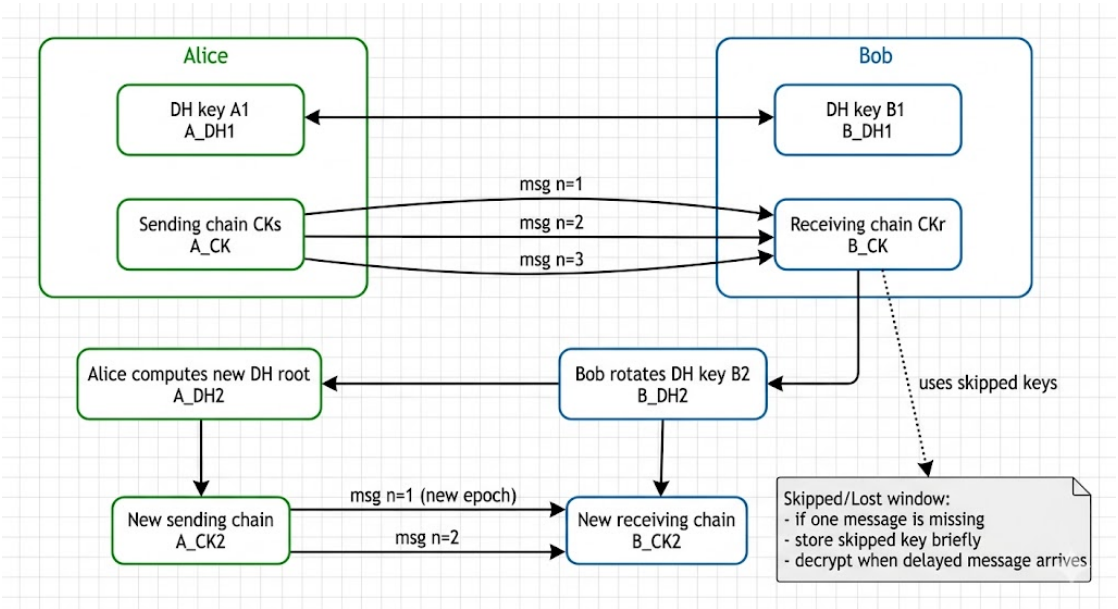
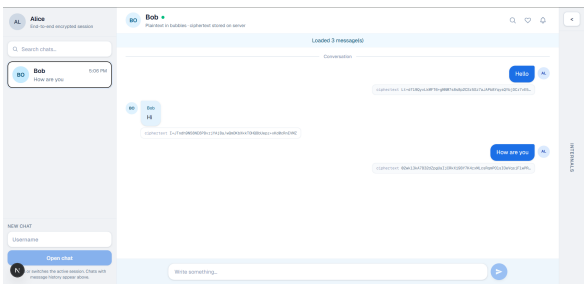


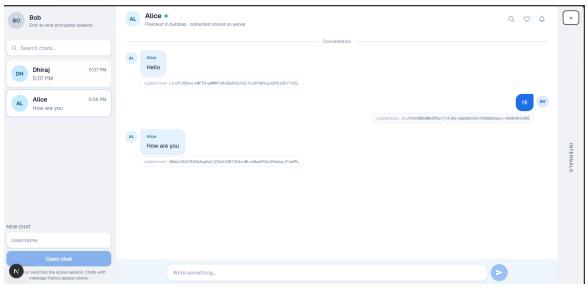
Figure 4: Double Ratchet progression across DH epochs and symmetric message-chain updates.

4 Expected behavior

From a user perspective, two independent browser sessions register as distinct participants, either may open a chat to the other. The opening encrypted message transports the X3DH header alongside ciphertext. Subsequent messages remain confidential to the endpoints and update ratchet-visible summaries where exposed. Reloading either session reloads ciphertext history from the relay and rehydrates local ratchet state so the thread remains usable across restarts, asynchronous delivery does not require both parties online simultaneously for storage, only eventual fetch.



(a) Alice to Bob encrypted messaging view



(b) Bob to Alice encrypted messaging view

Figure 5: Bidirectional messaging between both participants.

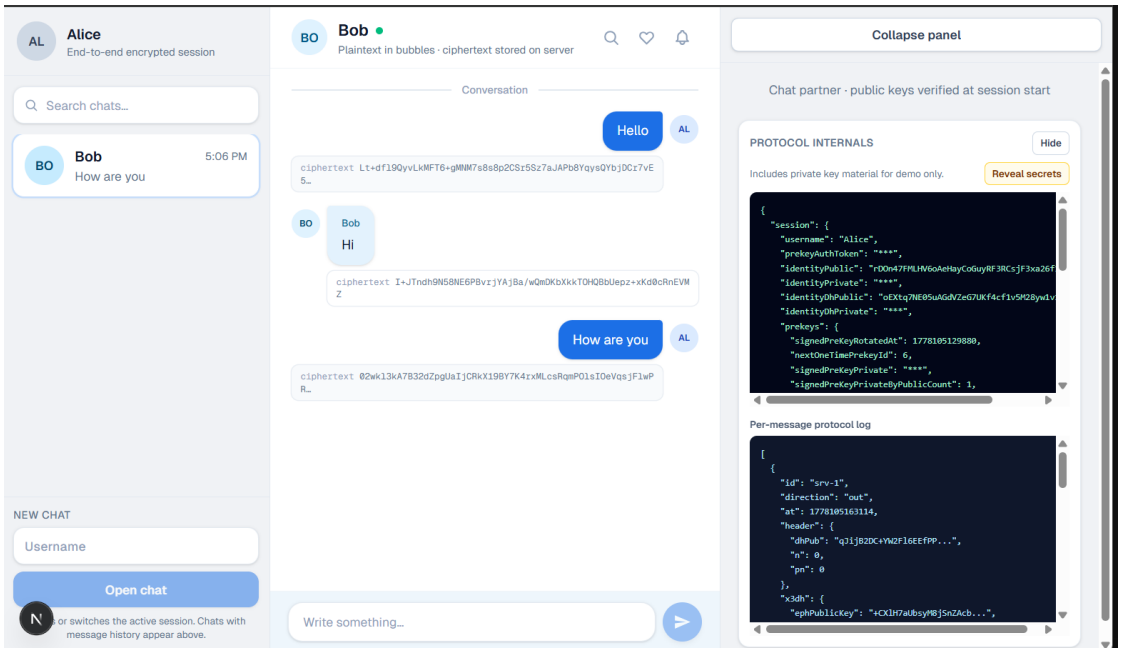


Figure 6: Protocol internals view showing stored handshake and ratchet-related state in the application workflow.

5 Security properties

Under the coursework threat model (honest-but-curious relay regarding content, uncompromised clients):

- **End-to-end confidentiality:** The relay does not possess keys required to unwrap message plaintext.

- **Pre-key authenticity:** The signed prekey is validated before DH use, binding medium-term keys to the long-term identity.
- **One-time prekey consumption:** Reserve-style handshakes deplete optional keys to limit reuse across initiations.
- **Forward-secrecy-oriented key schedule:** Ratchet chains and DH epochs advance material so past keys are not indefinitely reused for future messages.
- **Integrity:** Authenticated encryption rejects forged or altered payloads.

These properties do not include strong user authentication, multi-device identity, group messaging, or metadata protection (see Section 7).

6 Validation approach

Validation. Vitest exercises X3DH agreement, ratchet transitions, skip handling, and related invariants in isolation. ESLint enforces TypeScript hygiene. A production build of the Next.js application checks bundling and type integration. Complementary HTTP-level checks against a live API instance can confirm registration, bundle resolution, send, and read paths end-to-end.

7 Limitations and Future Work

7.1 Limitations

- Identities are lightweight and not backed by strong account authentication or recovery.
- The current scope is pairwise chat only, group messaging is out of scope.
- Browser storage for private keys is suitable for a demo, but weaker than secure hardware-backed storage.
- The relay can still observe social and traffic metadata such as communication pairs, timing, and message sizes.

7.2 Future work

- Add stronger identity and account security mechanisms.
- Improve key protection with secure-storage abstractions and non-exportable key workflows.
- Extend the protocol layer toward group communication.
- Add richer attack simulations and end-to-end regression scenarios.

8 How to run the project

1. Install prerequisites: Node.js (with npm) and Python 3.11 or newer.
2. Set up and start the backend service (create/activate virtual environment, install dependencies, run FastAPI/Uvicorn on port 3001).
3. Set up and start the frontend service (install Node dependencies, run Next.js on port 4000).
4. Open two browser sessions, register two users, and exchange messages in both directions.
5. Verify behavior: first message performs X3DH alignment, subsequent messages follow Double Ratchet updates.

Project repository: `https://github.com/Dhiraj455/SignalProtocol`
Demo video: `https://drive.google.com/file/d/1HLwxTmqWuUQwoyUhdaXwjWOSIvxNmaPX/view?usp=sharing`

9 Conclusion

The system demonstrates that X3DH and Double Ratchet ideas transfer cleanly into a compact browser-plus-relay architecture: cryptography and plaintext stay on the client, while FastAPI and JSON persistence confine server responsibility to public keys and ciphertext carriage. Transparency in both implementation (hand-written protocol logic atop standard primitives) and validation (unit, lint, build) matches the pedagogical aim of bridging protocol concepts and running software.